þÑþ•þ®þ» þ•þŽþ´þ£ (Hesab Sarraf)

Emergency Backup — Full Architecture & Documentation

Version: v01 — Emergency Backup

Date: 2026-10-23T08:45:28.107Z

Domain: https://motherapp.base44.app

Description: ûýþßþŽþã ûüþ®ûÿû” þµþ-þ•þ°û” þí þåþŽûþþ®þ˜þ¸þã ûüþ-þ•þªþ‘þŽþ´þ£ þ¯þ-þ• þÞûþþªþ’þ— ûþþÓþ•þ®þ» þ•þŽ

þæûþþ• :þ-þ•þªþ¸þë PDF þªþèþ˜þ´þë ûýþ˜þ³þ© ûýþ³þ-þ®þ‘ þªþèþãþ¯þŽûÿþç þ®ûþþ¯ þ©þ-þ•þîþã .þ–þ³þ• þéþªþ· þªûÿþßþîþ— þé

- ûýþ³þ-þ®þ‘ þªþèþãþ¯þŽûÿþç — þŽûþþîûX þéþ©þ•þ© (þÝþíþ•þªþŸ ûüþŽþëþ©þ-þîû•þ- þÖûÿþ×þ© þ©þ•þªþÌþ—)
- þ-þ•þ®þØþ˜þ³þ• þ–ûÿþÌþ¿þí RPCþ-þ© þŽþë Supabase (ûýþ³þ-þ®þ‘ þªþèþãþ¯þŽûÿþç)
- þÖûÿþ×þ© ûüþŽþë þ–þ³þŽûÿþ³ Storage (ûýþ³þ-þ®þ‘ þªþèþãþ¯þŽûÿþç)
- ûýþ³þ-þ®þ‘ þªþèþãþ¯þŽûÿþç (þÝþŽþÌþÓ ûýþÄûÿþ¤þã ûüþŽþëþ®ûÿþÐþ˜þã)

Table of Contents

1. Project Overview & Tech Stack
2. Architecture Diagram
3. Login & Authentication Flow
4. Data Flow Diagram
5. Database Schema & Relationships
6. RLS & Security Diagram
7. Financial Logic Diagram
8. File Dependency Map
9. Base44 Dependencies
10. Supabase Dependencies
11. Migration from Base44
12. Rebuild from Scratch
13. Frontend -> API -> Backend -> Database
14. Routes (React Router)
15. Entities
16. Security Model
17. Financial Logic
18. PWA & Offline
19. Environment Variables (no secrets)
20. Build & Deployment

1. Project Overview & Tech Stack

Project: þÑþ•þ®þ» þ•þŽþ´þ£ (Hesab Sarraf)

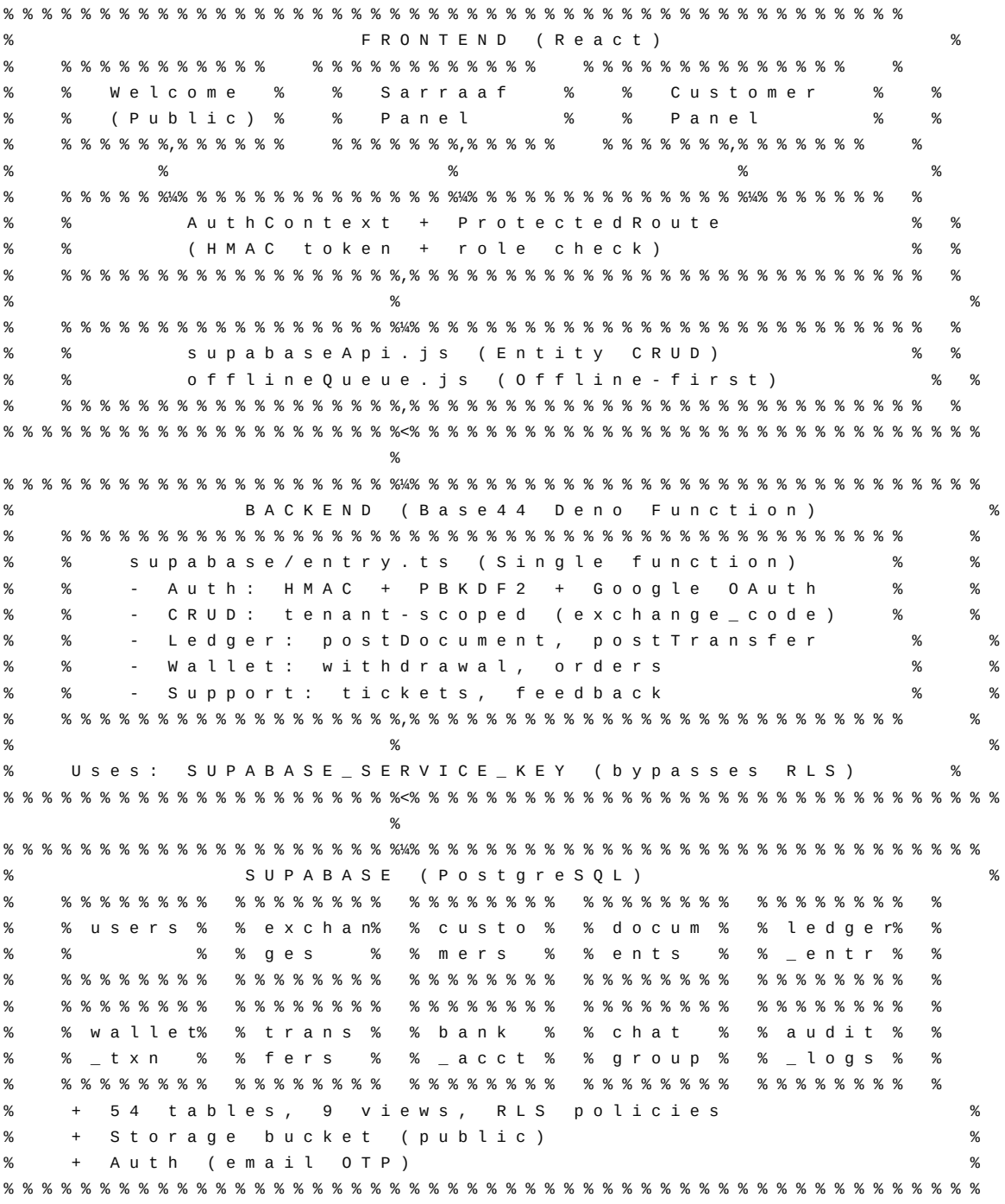
Status: In Development (Emergency Snapshot)

Frontend | React 18.2 + Vite 6.1
Styling | Tailwind CSS 3.4 + shadcn/ui + Radix UI
Routing | React Router DOM 6.26
Data Fetching | @tanstack/react-query 5.84
Forms | react-hook-form 7.54 + zod 3.24
Icons | lucide-react 0.475
Charts | recharts 2.15
Animation | framer-motion 11.16
Drag & Drop | @hello-pangea/dnd 17
Maps | react-leaflet 4.2
3D | three.js 0.171

PDF | jspdf 4.2 + html2canvas 1.4
Database | PostgreSQL (Supabase)
Backend Runtime | Deno (Base44 Backend Functions)
Auth | Custom HMAC + PBKDF2 + Supabase Auth OTP + Google OAuth
Payment | Stripe (installed, disabled)
File Storage | Supabase Storage (public bucket)
Supabase SDK | @supabase/supabase-js 2.109
Base44 SDK | @base44/sdk 0.8.51

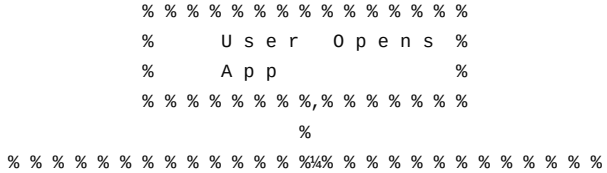
---

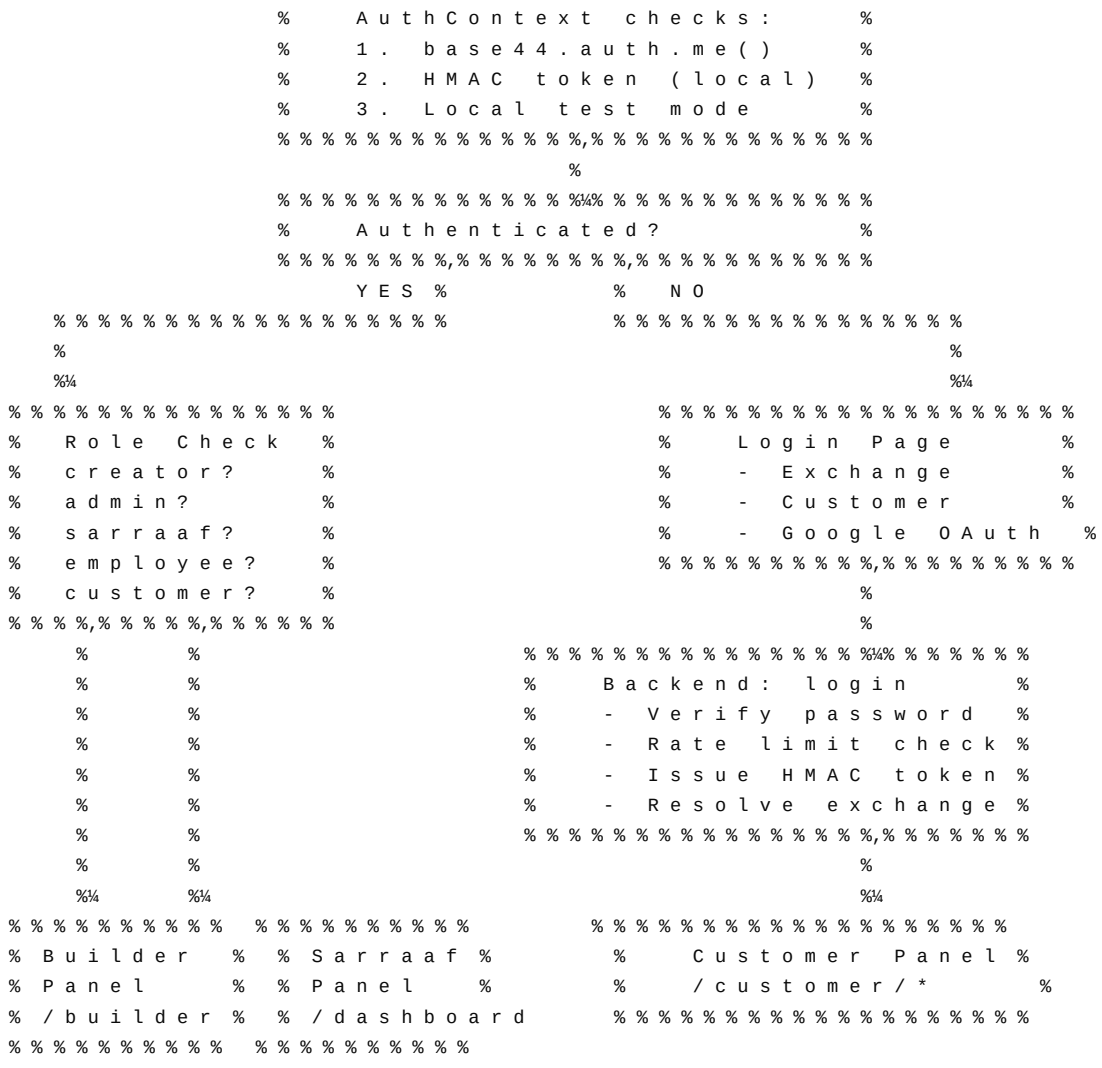
2. Architecture Diagram



---

3. Login & Authentication Flow





4. Data Flow Diagram



5. Database Schema & Relationships

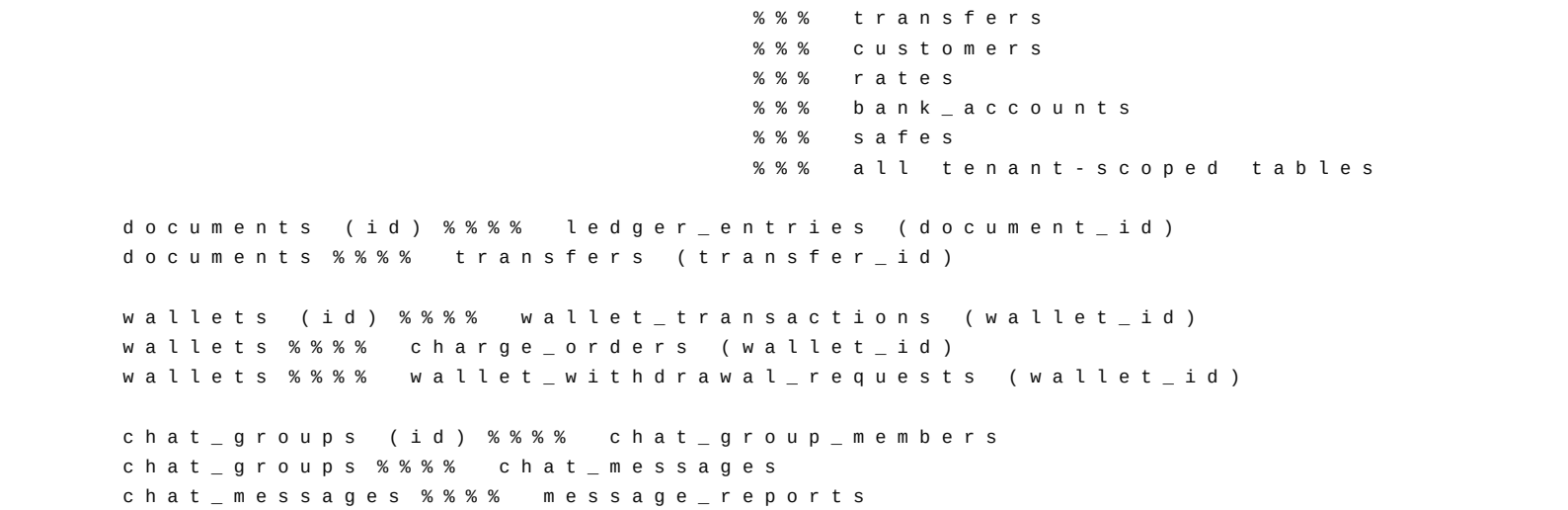
Tables (54+):

users
exchanges
customers
documents
transfers
conversions
treasury_transactions
bank_accounts
bank_transactions
journal_entries
ledger_entries
rates
idempotency_keys
employees
origins
safes
rules
feature_settings
app_settings
audit_logs
wallet_transactions
charge_orders
online_requests
online_products
wallets
wallet_withdrawal_requests
wallet_topup_requests
wallet_bans
customer_notifications
customer_transfer_requests
customer_links
email_otps
user_notifications
user_permissions
user_feedback
support_tickets
support_messages
error_reports
bank_gateways
payment_transactions
chat_groups
chat_group_members
chat_messages
message_reports
sellers
packages
pricing_plans
plan_quotas
quota_change_logs
referrals
agency_links
backup_records
rate_settings
text_labels

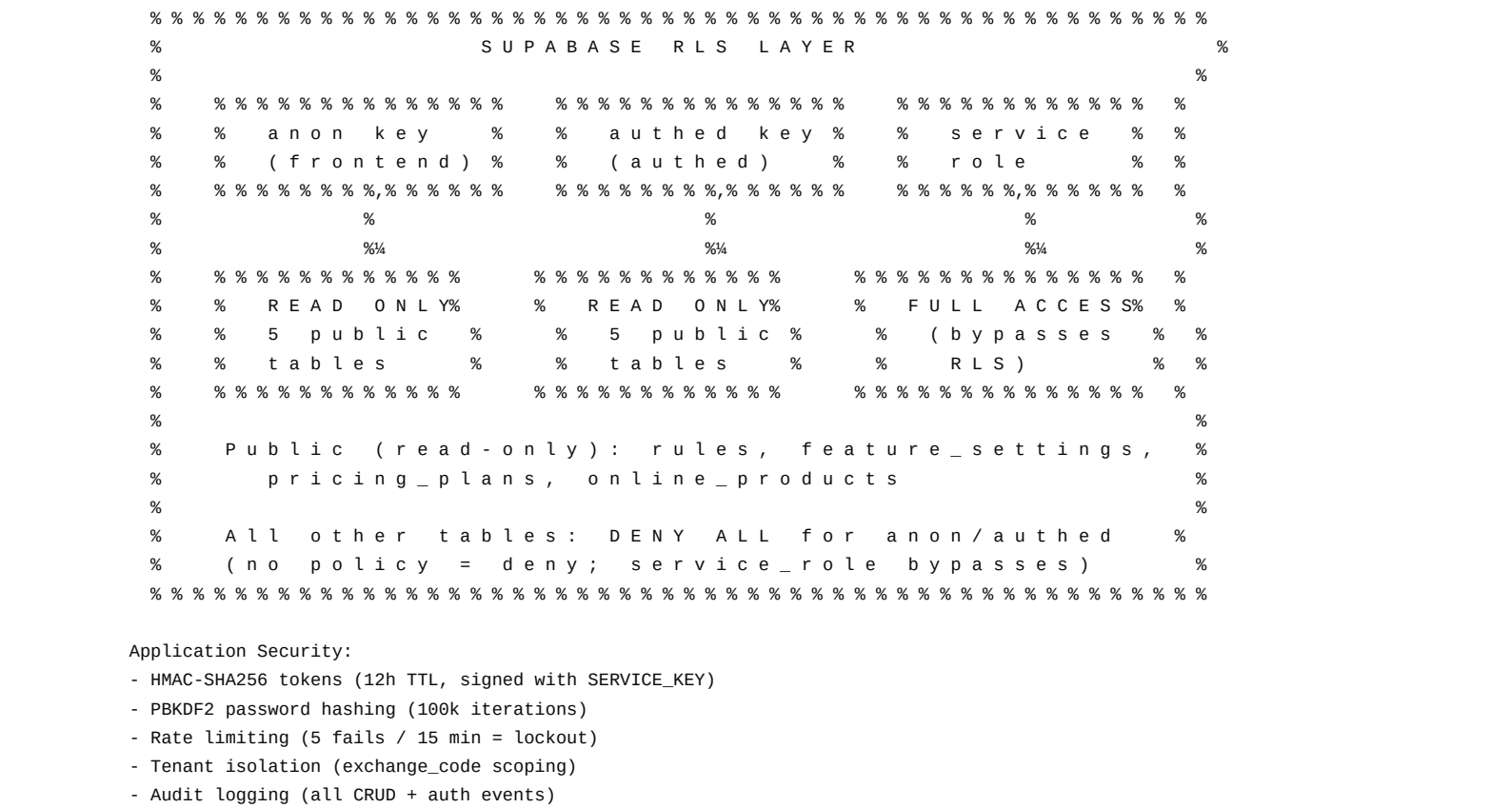
Key Relationships

```
u s e r s  ( i d )  % % %,% %   e x c h a n g e s  ( o w n e r _ u s e r _ i d )
                    % % %   c u s t o m e r s  ( u s e r _ i d )
                    % % %   w a l l e t s  ( u s e r _ i d )

e x c h a n g e s  ( e x c h a n g e _ c o d e )  % % %,% %   d o c u m e n t s
```

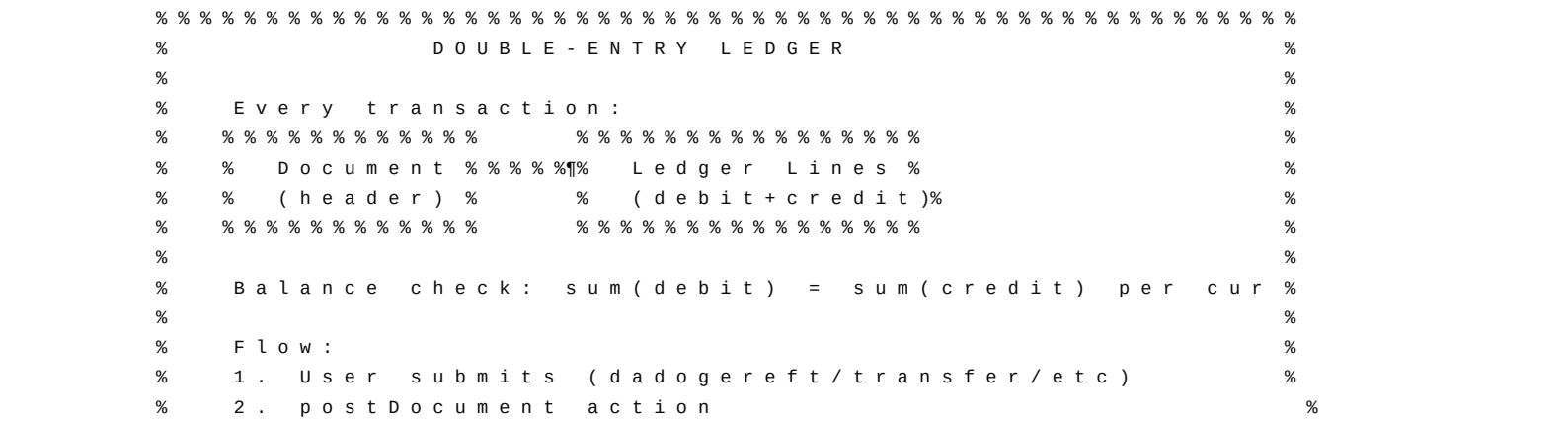


6. RLS & Security Diagram



Application Security:
- HMAC-SHA256 tokens (12h TTL, signed with SERVICE_KEY)
- PBKDF2 password hashing (100k iterations)
- Rate limiting (5 fails / 15 min = lockout)
- Tenant isolation (exchange_code scoping)
- Audit logging (all CRUD + auth events)

7. Financial Logic Diagram



```
%     3.  Validate  double-entry  balance                          %
%     4.  Idempotency  check  (client_operation_id)               %
%     5.  Generate  doc_number  (DOC-000001)                      %
%     6.  Insert  document  header                                %
%     7.  Insert  ledger_entries  (debit  +  credit)               %
%     8.  If  lines  fail !'  rollback  document                  %
%     9.  Record  idempotency  key                                 %
%     10.  Audit  log                                              %
%                                                                 %
%     Wallet  Operations:                                         %
%     -  Read  wallet.balance                                     %
%     -  Check  newBalance  >=  0                                 %
%     -  UPDATE  ...  WHERE  balance  =  currentBalance           %
%        (optimistic  lock  prevents  concurrent  overspend)%
%%%
```

base44/shared/rls_policies.sql | — | RLS policies SQL
base44/config.jsonc | — | Base44 app configuration
base44/entities/*.jsonc | — | Entity schema definitions
src/lib/seedBase44Entities.js | — | Entity seeding utility

Migration notes:
- base44.auth.* !' Replace with Supabase Auth or custom JWT
- base44.entities.* !' Replace with supabaseApi.js (already done)
- base44.functions.invoke !' Replace with direct Supabase calls or Edge Functions
- base44.integrations.Core.* !' Replace with direct API calls
- @base44/vite-plugin !' Remove from vite.config.js
- index.html <head> !' Remove Base44 runtime script injection

10. Supabase Dependencies

Real Supabase connections, tables, and services:

@supabase/supabase-js | 2.109.0 | Supabase JavaScript client
src/lib/supabaseClient.js | — | Direct Supabase client (VITE_ env vars)
src/lib/supabaseApi.js | — | Supabase API layer (entity CRUD + field mapping)
src/lib/supabaseDB.js | — | Direct Supabase interaction module
src/lib/supabaseAuth.js | — | Supabase auth helpers
base44/functions/supabase/entry.ts | — | Backend function using service_role key
VITE_SUPABASE_URL | — | Frontend Supabase URL (public, safe)
VITE_SUPABASE_KEY | — | Frontend Supabase anon key (public, RLS-protected)
SUPABASE_URL | — | Backend Supabase URL (server-side)
SUPABASE_KEY | — | Backend Supabase anon key (server-side)
SUPABASE_SERVICE_KEY | — | Backend service role key (bypasses RLS)
Tables (54+) | — | See Supabase Structure section
RLS Policies | — | See RLS section — defense in depth
Auth (OTP) | — | Supabase built-in email OTP for verification
Storage | — | Public bucket for file uploads (receipts)

Tables (54+): users, exchanges, customers, documents, transfers, conversions, treasury_transactions, bank_accounts, bank_transactions, journal_entries, ledger_entries, rates, idempotency_keys, employees, origins, safes, rules, feature_settings, app_settings, audit_logs, wallet_transactions, charge_orders, online_requests, online_products, wallets, wallet_withdrawal_requests, wallet_topup_requests, wallet_bans, customer_notifications, customer_transfer_requests, customer_links, email_otps, user_notifications, user_permissions, user_feedback, support_tickets, support_messages, error_reports, bank_gateways, payment_transactions, chat_groups, chat_group_members, chat_messages, message_reports, sellers, packages, pricing_plans, plan_quotas, quota_change_logs, referrals, agency_links, backup_records, rate_settings, text_labels

Views: v_treasury_balances, v_bank_balances, v_customer_balances, v_wallet_balances, v_agent_balances, v_profit_loss, v_currency_balances, v_exchange_summary, v_transactions

RPC Functions: post_transfer_transaction, reverse_document_transaction, edit_document_transaction, get_account_balance

11. Migration from Base44

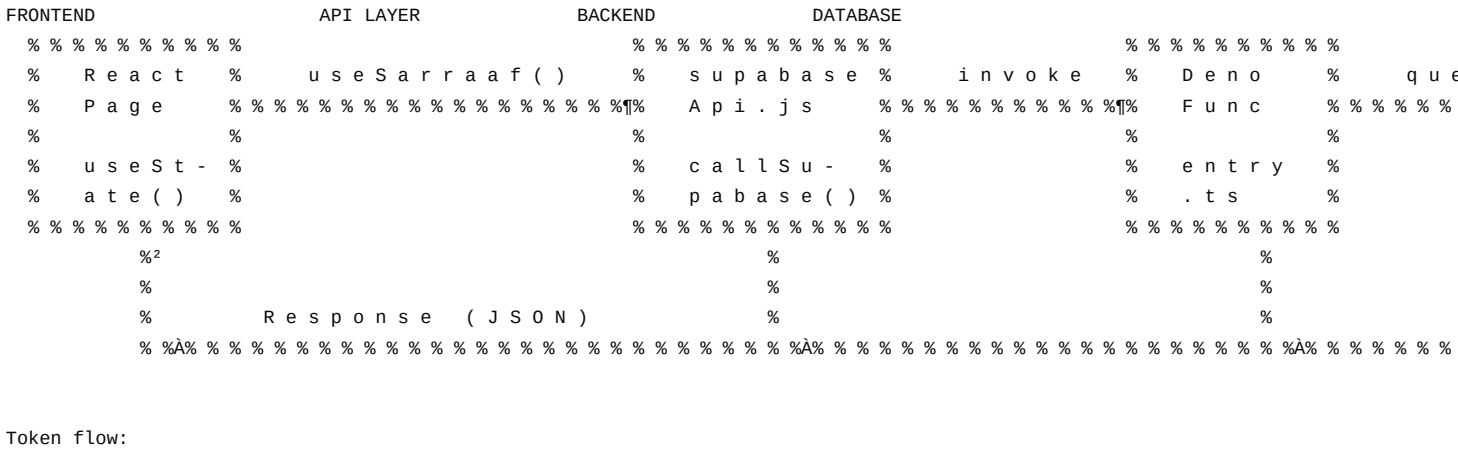
Step 1: Replace base44.auth with Supabase Auth or custom JWT
Step 2: Replace base44.entities with supabaseApi.js (already done)
Step 3: Replace base44.functions.invoke with direct Supabase calls
Step 4: Replace base44.integrations with direct API implementations
Step 5: Remove @base44/vite-plugin from vite.config.js
Step 6: Remove Base44 runtime script from index.html <head>
Step 7: Deploy backend function as Supabase Edge Function
Step 8: Set environment variables (SUPABASE_URL, SUPABASE_SERVICE_KEY)
Step 9: Test all CRUD operations via supabaseApi.js

Step 10: Verify RLS policies are active

12. Rebuild from Scratch

Step 1: Create new Vite + React project
Step 2: Install dependencies (see package.json)
Step 3: Set up Tailwind CSS + shadcn/ui
Step 4: Create Supabase project + run schema SQL
Step 5: Apply RLS policies SQL
Step 6: Create backend function (copy entry.ts)
Step 7: Set secrets (SUPABASE_URL, SUPABASE_SERVICE_KEY, CREATOR_EMAIL, CREATOR_PASSWORD)
Step 8: Copy source files (src/*)
Step 9: Update routing (App.jsx)
Step 10: Test auth + CRUD + financial operations

13. Frontend -> API -> Backend -> Database



14. Routes (React Router)

/ | Welcome | Public
/setup-database | SetupDatabase | Public
/login | Login | Public
/register | Register | Public
/forgot-password | ForgotPassword | Public
/reset-password | ResetPassword | Public
/dashboard | Dashboard | Sarraaf
/customers | Customers | Sarraaf
/customers/:id | CustomerDetail | Sarraaf
/dadogereft | Dadogereft | Sarraaf
/trade | Trade | Sarraaf
/transfers/:direction | Transfer | Sarraaf
/cash | Cash | Sarraaf
/bank | Bank | Sarraaf
/currencies | Currencies | Sarraaf
/reports | Reports | Sarraaf
/profit-loss | ProfitLoss | Sarraaf
/settings | Settings | Sarraaf
/transfer-book | TransferBook | Sarraaf
/journal | Journal | Sarraaf
/general-ledger | GeneralLedger | Sarraaf
/journal-entry | JournalEntry | Sarraaf
/balance-sheet | BalanceSheet | Sarraaf
/reconciliation | Reconciliation | Sarraaf
/convert | CurrencyConvert | Sarraaf
/online-products | OnlineProducts | Sarraaf

/payment-gateway | PaymentGateway | Sarraaf
/agents | Agents | Sarraaf
/debtors | Debtors | Sarraaf
/notifications | Notifications | Sarraaf
/agency-network | AgencyNetwork | Sarraaf
/chat | ChatGroups | Sarraaf
/chat/group/:id | ChatGroupDetail | Sarraaf
/sarraaf-market | SarraafMarket | Sarraaf
/sarraaf-chat | SarraafChat | Sarraaf
/backup | Backup | Sarraaf
/support | SarraafSupport | Sarraaf
/builder | BuilderPanel | Creator/Admin
/main-settings | MainSettings | Creator/Admin
/customer | CustomerHome | Customer
/customer/wallet | CustomerWallet | Customer
/customer/recharge | CustomerRecharge | Customer
/customer/games | CustomerGames | Customer
/customer/transfer | CustomerTransfer | Customer
/customer/withdraw | CustomerWithdraw | Customer
/customer/support | CustomerSupport | Customer
/customer/account | CustomerAccount | Customer
/customer/orders | CustomerOrders | Customer
/customer/messages | CustomerMessages | Customer
/customer/notifications | CustomerNotifications | Customer
/customer/profile | CustomerProfile | Customer
/customer/chat | CustomerChat | Customer
/receipt | Receipt | Authenticated

15. Entities

Entity schemas defined in base44/entities/*.jsonc:

User
SarraafProfile
Customer
Wallet
Transaction
Transfer
Agent
Currency
BankAccount
BankGateway
Safe
OnlineProduct
OnlineOrder
ChatGroup
GroupMember
GroupMessage
MessageReport
WalletTransaction
WalletTopupRequest
WalletWithdrawalRequest
WalletBan
CustomerTransferRequest
CustomerNotification
CustomerLink
PaymentTransaction
PaymentRecord
AuditLog
AppSetting
FeatureSetting
PricingPlan
PlanQuota
QuotaChangeLog
UserPermission
UserNotification
UserFeedback

ErrorReport
SupportTicket
SupportMessage
Announcement
Referral
AgencyLink
Seller
Package
RateSetting
TextLabel
BackupRecord
BuilderAccess
CustomerChat
LedgerEntry

16. Security Model

- HMAC-SHA256 stateless session tokens (12h TTL)
- PBKDF2 password hashing (100k iterations, SHA-256)
- Rate limiting: 5 failed logins in 15 min = lockout
- Tenant isolation: exchange_code scoping on all exchange-scoped tables
- RLS: defense in depth — anon key can only read public tables
- Service role: bypasses RLS (backend only, never exposed)
- Google OAuth: creates customer with placeholder mobile
- Audit logging: all login, CRUD, password change events
- No secrets in frontend: VITE_ vars are public, RLS-protected
- Idempotency keys: prevent duplicate financial operations

17. Financial Logic

- Double-entry ledger: every transaction posts debit + credit lines
- Atomic document posting: documents + ledger_entries in single operation
- Idempotency: client_operation_id prevents duplicate operations
- Wallet operations: optimistic-lock balance updates (eq on current balance)
- Transfer posting: RPC post_transfer_transaction (atomic)
- Document reversal: RPC reverse_document_transaction
- Document edit: RPC edit_document_transaction (soft-delete old + insert new)
- Balances: SQL views (v_treasury_balances, v_bank_balances, v_customer_balances, etc.)
- Withdrawal: wallet deduction + withdrawal request + wallet_transaction
- Customer order: product price validation + wallet deduction + order creation

18. PWA & Offline

- Service Worker: public/sw.js (needs update for full PWA)
- Manifest: public/manifest.json
- Offline Queue: src/lib/offlineQueue.js (queues writes, syncs on reconnect)
- Cache: localStorage-based caching for offline reads
- Sync Status: src/components/SyncIndicator.jsx
- Keep Alive: src/lib/useKeepAlive.js (prevents Supabase inactivity pause)

19. Environment Variables (no secrets)

Frontend (public, safe — RLS protected):
  VITE_SUPABASE_URL — Supabase project URL
  VITE_SUPABASE_KEY — Supabase anon key (public, RLS-protected)

Backend (server-side, never exposed):

SUPABASE_URL — Supabase project URL
SUPABASE_KEY — Supabase anon key
SUPABASE_SERVICE_KEY — Service role key (bypasses RLS)
CREATOR_EMAIL — Creator account email
CREATOR_PASSWORD — Creator account password
BASE44_APP_ID — Base44 app ID (for SDK fallback)

NOTE: All secrets are set via Base44 Secrets dashboard, not in code.

---

20. Build & Deployment

Build: vite build (Vite + React)
Deploy: Base44 platform (auto-deploy on push)
Backend: Deno runtime (base44/functions/supabase/entry.ts)
Database: Supabase PostgreSQL (managed)
Storage: Supabase Storage (public bucket)
Mobile: Base44 native build (iOS/Android)
Custom Domain: Configure in Base44 dashboard

This PDF was generated as an emergency backup snapshot.
Generated: 2026-10-23T08:45:28.107Z
Ｐｒｏｊｅｃｔ: þÑþ•þ®þ» þ•þŽþ´þ£ （Ｈｅｓａｂ Ｓａｒｒａｆ）
Version: v01 — Emergency Backup

No secrets, passwords, or API keys are included in this document.
Use the ZIP backup for source code and the Supabase backup for database structure.